

Recurrence Relations

Recurrence relations describe the running time of recursive algorithms by expressing $T(n)$ in terms of smaller inputs.

Natural Explanation

When an algorithm calls itself, total work comes from: - Work done inside recursive calls. - Extra work done outside recursion (splitting, merging, comparisons, etc.).

A recurrence is just an equation that captures this pattern.

Example intuition: - Binary search keeps one half each step, so problem size shrinks fast. - Merge sort solves two halves and then merges, so each level does linear work.

Academic Form

A recurrence has the form:

$$T(n) = (\text{recursive part}) + (\text{non-recursive part}).$$

Common divide-and-conquer template:

$$T(n) = aT(n/b) + f(n), \quad T(1) = \Theta(1),$$

where: - a = number of subproblems, - n/b = subproblem size, - $f(n)$ = combine/split overhead.

Why We Use Them

- They turn recursive code into analyzable math.
 - They let us derive asymptotic bounds like $O(\log n)$ and $O(n \log n)$.
 - They connect algorithm design (divide/conquer/combine) to complexity classes.
-

Standard Examples

Binary Search

$$T(n) = T(n/2) + O(1) \Rightarrow T(n) = O(\log n).$$

Reason: each step halves the remaining interval, and only constant work is done per step.

Merge Sort

$$T(n) = 2T(n/2) + O(n) \Rightarrow T(n) = O(n \log n).$$

Reason: about $\log n$ levels of recursion, each doing total $O(n)$ merge work.

Common Solving Methods

- Iteration / expansion (unroll the recurrence).
- Substitution (guess-and-prove bound by induction).
- Master Theorem (for $T(n) = aT(n/b) + f(n)$ forms).

For this course level, expansion + recursion-tree intuition is often enough for common recurrences.

Pitfalls

- Forgetting base cases.
 - Counting only recursive calls and forgetting combine work.
 - Mixing exact equalities with asymptotic bounds incorrectly.
-

Related

- [[Divide and Conquer]]
- [Merge Sort]
- [Binary Search]
- [[Big-O, Big-Omega, Big-Theta]]
- [[Lectures/Week 2]]